ResumeIQ — NLP Resume Analyzer

Technical Analysis Report

Team: Siddhank | Anurag | Himanshu

May 2026

---

1. Executive Summary

ResumeIQ is a full-stack, AI-powered web application that automates the process of matching resumes to job descriptions using multiple NLP signals. It addresses a real-world problem: candidates submitting generic resumes to Applicant Tracking Systems (ATS) with little insight into how well their profile aligns with a job posting.

The system is built on Python and Streamlit, integrating several NLP libraries — Sentence-BERT, TF-IDF via scikit-learn, spaCy, and a curated 270+ skill taxonomy — to produce a composite match score and detailed, actionable feedback. It also offers an AI-assisted resume rewriting feature powered by OpenRouter, and can export a formatted PDF report of the analysis.

Key highlights of the system include:

- Multi-signal composite scoring: Semantic Similarity (SBERT), TF-IDF Keyword Coverage, Skill Taxonomy Matching, and ATS Friendliness scoring.
- A curated taxonomy of 270+ skills across 10 professional categories with alias resolution and false-positive filtering.
- Semantic skill matching using cosine similarity embeddings at a calibrated threshold (0.82) to reduce false positives.
- An ATS scoring model covering 9 dimensions including action verbs, quantified achievements, section structure, and formatting safety.
- AI-assisted resume rewriting via OpenRouter with multi-model fallback support.
- Interactive Streamlit UI with radar charts, keyword tags, sentence-level matching, and downloadable PDF reports.

Target users include job seekers, career counselors, HR professionals, and recruitment automation teams.

2. Project Overview and Architecture

2.1 Project Goals

ResumeIQ was designed with the following primary objectives:

- Automate resume-JD gap analysis without human recruiters, making the process fast, repeatable, and quantitative.
- Combine multiple NLP signals to produce a composite match score more reliable than any single metric alone.

- Surface actionable, category-aware recommendations so users know exactly what to improve.
- Maintain ATS awareness — addressing not just semantic fit, but also structural and formatting compatibility.
- Offer an AI rewriting assistant to help users implement suggestions immediately within the tool.

2.2 High-Level Architecture

The application follows a modular three-layer architecture, as described below:

Layer	Module	Responsibility
Presentation	app.py	Streamlit UI, user input handling, results visualization (radar charts, keyword tags, metric cards)
Business Logic	utils.py	All NLP computations: SBERT similarity, TF-IDF, skill matching, ATS scoring, NER, PDF generation, AI rewriting
Data / Config	skills.py	Skill taxonomy (270+ skills, 10 categories), alias map, false-positive pairs, helper functions
Dependencies	requirements.txt	Library declarations: streamlit, sentence-transformers, scikit-learn, spaCy, pdfplumber, plotly, fpdf2, openai

2.3 Data Flow

The user journey through the system follows a sequential pipeline with five stages:

- Step 1 — Input: The user provides a resume (text paste or PDF upload) and job description via the Streamlit sidebar. pdfplumber handles PDF extraction with a fallback error message for image-based PDFs.
- Step 2 — NLP Analysis: Upon clicking Analyze, four parallel analysis streams execute: (a) SBERT semantic similarity, (b) TF-IDF keyword extraction and coverage scoring, (c) skill taxonomy extraction and semantic skill matching, and (d) ATS compatibility scoring with spaCy NER.
- Step 3 — Score Aggregation: calculate_composite_score() combines the four signals using dynamically adjusted weights based on job type (tech vs. non-tech).
- Step 4 — Results Rendering: Results are displayed across five tabs — Overview, Skills, Keywords, ATS Health, and Suggestions & AI — with interactive Plotly visualizations.
- Step 5 — Output: Users can download a formatted PDF report or invoke the AI resume rewriter powered by OpenRouter.

3. NLP Pipeline — Core Algorithms

3.1 Semantic Similarity (Sentence-BERT)

Semantic similarity is the anchor signal of the composite score, contributing 35% for technical roles and 45% for non-technical roles. The implementation uses the all-MiniLM-L6-v2 Sentence Transformer model, cached via Streamlit's @st.cache_resource decorator to avoid repeated initialization overhead.

The algorithm operates through three steps:

- Text Preprocessing: Both texts are lightly cleaned using clean_text(), which only normalizes whitespace. Punctuation and casing are preserved intentionally, as SBERT relies on full linguistic context for accurate semantic encoding.
- Smart Chunking: Long texts are split into beginning, middle, and end thirds (each approximately 133 words) via _chunk_text(max_words=400). This ensures work experience content — typically in the middle of a resume — is not truncated silently.
- Embedding and Cosine Similarity: Both processed texts are encoded into 384-dimensional dense vectors using model.encode(). Cosine similarity between the two vectors is computed using sklearn and scaled to a 0–100 percentage.

Design Note: Light cleaning (whitespace only) was chosen over aggressive normalization (lowercasing, stemming) because SBERT's transformer architecture extracts richer contextual embeddings from natural language. Heavy pre-processing degrades SBERT performance.

3.2 TF-IDF Keyword Analysis

The keyword analysis component identifies high-importance terms from the job description that appear — exactly or semantically — in the resume. It contributes 25% (tech roles) or 40% (non-tech roles) to the composite score.

The algorithm consists of four steps:

- Keyword Extraction: extract_tfidf_keywords() splits both texts into sentences, builds a TF-IDF vectorizer with a joint corpus for proper cross-document IDF weighting, and extracts top N keywords using ngram_range=(1,2) to capture multi-word technical terms.
- Token Pattern: A custom regex pattern filters tokens to those starting with a letter and at least 2 characters long, eliminating numerical noise and single-character tokens.
- Exact Match: Word-boundary regex is used instead of plain substring matching to prevent partial matches — for example, 'node' should not match 'nodejs'.
- Semantic Fallback: If exact match fails, the JD keyword's embedding is compared against all resume keyword embeddings. A match is recorded if cosine similarity exceeds 0.82.

Coverage score interpretation:

Coverage %	Verdict	Recommended Action
70%+	Strong keyword alignment	Minor additions only
40% – 69%	Moderate alignment	Add 5–8 JD keywords to resume

Below 40%	Weak alignment	Significant keyword gap — major revision needed

4. Skill Taxonomy and Matching Engine

4.1 Skill Taxonomy Design

The skill taxonomy (skills.py) serves as the knowledge backbone of the system. It contains over 270 curated skills organized into 10 professional categories:

Category	Skill Count	Examples
Programming Languages	35	Python, Java, TypeScript, Rust, Go, SQL, Kotlin
Web & Frontend	44	React, Angular, Next.js, Tailwind CSS, GraphQL, Cypress
Backend & Frameworks	38	Node.js, Django, FastAPI, Spring Boot, gRPC, NestJS
Data & Databases	36	PostgreSQL, MongoDB, Redis, Snowflake, Vector Databases
AI & Machine Learning	55	TensorFlow, PyTorch, LangChain, RAG, LLMs, BERT, YOLO
Cloud & DevOps	54	AWS, Docker, Kubernetes, Terraform, GitHub Actions, ArgoCD
Data Engineering	34	Apache Spark, Kafka, Airflow, dbt, Databricks, BigQuery
Version Control & Collaboration	21	Git, Jira, Agile, Scrum, TDD, Code Review
Security & Testing	22	Pytest, Selenium, OAuth2, OWASP, Penetration Testing
Soft Skills	20	Leadership, Communication, Stakeholder Management, Mentoring

4.2 Alias Resolution and False Positive Filtering

Two data structures prevent common matching errors that degrade accuracy:

- SKILL_ALIASES (14 pairs): Maps variant names to canonical forms. Applied before comparison so 'reactjs', 'react.js', and 'react' are treated identically. Also covers: nodejs, golang, sklearn, postgres, huggingface, rails, k8s, powerbi.

- SKILL_FALSE_POSITIVES (14 pairs): Bidirectional frozenset pairs explicitly blocked from cross-matching. Covers high-risk confusions such as Java/JavaScript, Java/TypeScript, Flask/Spark, R/Rust, C/C++, SQL/NoSQL, Go/Groovy, Python/PyTorch, Scala/SQL, and others.

4.3 Semantic Skill Matching Algorithm

get_semantic_skill_matches() operates in three sequential priority stages:

- Stage 1 — Exact/Alias Match: Both the JD skill and its normalized alias are checked against the set of normalized resume skills. This is an O(1) lookup via Python sets.
- Stage 2 — Substring Containment: Word-boundary regex searches the raw resume text for the JD skill. This reliably catches multi-word skills like 'machine learning' or 'ruby on rails' that may appear in prose.
- Stage 3 — Semantic Embedding Match: SBERT embeddings of the JD skill are compared against all extracted resume skill embeddings. A match is recorded only if cosine similarity >= 0.82 AND the pair is not in SKILL_FALSE_POSITIVES. This threshold was raised from the original 0.75 to cut false positives such as 'AWS' matching 'Angular'.

Improvement Note: An earlier version harvested random uppercase abbreviations from the resume text as additional skill candidates. This was removed because it caused significant false-positive semantic matches.

5. ATS Compatibility Scoring Engine

5.1 Design Philosophy

The ATS scoring model (calculate_ats_score in utils.py) was redesigned from a 'start at 100 and deduct' model to an additive earn-your-points model. The original approach resulted in artificially high scores (~98) because deductions were minimal. Real ATS tools like Jobscan and Resume Worded score on 15+ dimensions with significant weights — this implementation mirrors that approach.

5.2 Scoring Dimensions (Total: 100 Points)

Dimension	Max Points	What It Checks
Contact Information	10	Email (4pts), phone (3pts), LinkedIn URL (2pts), GitHub URL (1pt)
Section Structure	20	Experience (7pts), Education (5pts), Skills (5pts), Summary (2pts), Projects (1pt)
Content Length	10	Sweet spot 300–800 words (10pts); penalizes very short (<200) or very long (>1200)
Action Verbs	15	65+ action verbs checked; full marks for 10+ distinct verbs found

Quantified Achievements	15	8 metric patterns: percentages, multipliers, dollar amounts, team sizes, time savings
Formatting Safety	10	Deducts for decorative symbols (>5), garbled characters, very long lines
Readability Signals	10	Date ranges (5pts), job title keywords (3pts), consistent past tense (2pts)
Professional Links	5	Bonus for portfolio/personal site beyond LinkedIn and GitHub
File/Text Quality	5	Alpha character ratio (low = image-based PDF), excess whitespace detection

5.3 Action Verb Detection

The system maintains a curated list of 65+ strong action verbs covering leadership (Led, Spearheaded, Directed), engineering (Architected, Engineered, Refactored), delivery (Launched, Deployed, Shipped), and impact (Improved, Optimized, Increased). Scoring is graduated as follows:

- 0 verbs: 0 points — issues flagged, tip provided
- 1–2 verbs: 4 points — low variety warning
- 3–5 verbs: 8 points — adequate, tip to improve
- 6–9 verbs: 12 points — good performance
- 10+ verbs: 15 points (full marks) — rich, varied vocabulary

5.4 Quantified Achievement Detection

Eight regex patterns check for measurable impact: percentage figures (40%), multipliers (3x), dollar amounts ($20K), user/team counts, scale words (million, billion), project counts, verb+number combinations (increased by 30%), and time savings (3 months). Each pattern hit contributes to a graduated score, with 7+ hits earning full marks.

6. Composite Scoring and Entity Recognition

6.1 Composite Score Formula

The final composite score combines four NLP signals using dynamically weighted summation. Weights are adjusted based on whether the job description is classified as technical (more than 3 hard skills detected) or non-technical:

Signal	Tech Role Weight	Non-Tech Weight
Semantic Similarity (SBERT)	35%	45%

Keyword Coverage (TF-IDF)	25%	40%
Skill Match %	30%	5%
ATS Friendliness Score	10%	10%

Guard Clause: If a JD has zero detected skills (non-technical role), skill_match_pct is treated as 100% to avoid unfairly penalizing candidates. The composite is clamped between 0 and 100.

Interpretation Bands:

- 75%+ — Excellent Match: Strong alignment across all signals.
- 50–74% — Good Match: Most requirements covered, targeted gaps exist.
- Below 50% — Needs Work: Significant revision required.

6.2 Named Entity Recognition (NER)

extract_resume_entities() uses a hybrid approach combining spaCy's en_core_web_sm model with targeted regex patterns for domain-specific entities:

Entity Type	Method	Pattern / Label
Company Names	spaCy	ORG entity label, min length 2, deduplicated, top 5
Person Names	spaCy	PERSON entity label, min length 2, top 2
Email Addresses	Regex	[\w.+-]+@[\w-]+\.[a-z]{2,}
Phone Numbers	Regex	Flexible international format with brackets and dashes
Degree / Education	Regex	5 patterns: B.Tech, M.Tech, PhD, MBA, Diploma, etc.
Years of Experience	Regex	N+ years of experience pattern
Job Titles	Keyword List	16 standard tech roles matched via substring
Certifications	Regex	AWS/Google/Microsoft Certified, CPA, CFA, PMP, CISSP, etc.

6.3 Section-Level Scoring

calculate_section_scores() produces granular scores for three resume sections:

- Skills Score: Directly maps to the semantic skill match percentage.
- Experience Score: Compares extracted years of experience from the resume against the JD requirement. Scoring is proportional — for example, 3 years vs. 5 years required yields 60%.

- Education Score: Binary check — if the JD specifies a degree and the resume has any degree listed, the score is 100%; if no degree is detected, the score is 0%.

7. User Interface, Visualizations, and AI Features

7.1 Streamlit UI Architecture

app.py implements a single-page Streamlit application. The UI is structured into five logical sections:

- Sidebar: Explains the tool's workflow and the dynamic score weighting model. Always visible, providing context as users interpret results.
- Input Zone: Two-column layout with Resume and Job Description inputs, each supporting text paste or PDF upload modes.
- Results Header (Hero Card): Large circular composite score display with color-coded verdict text and four metric mini-cards beneath.
- Analysis Tabs: Five tabbed panels — Overview, Skills, Keywords, ATS & Resume Health, and Suggestions & AI — for organized exploration of results.
- Footer: Tech stack attribution and model credits.

7.2 Visualizations

Visualization	Location	What It Shows
Skill Category Radar	Skills Tab	Spider chart of coverage % per JD-required skill category. Only categories with JD skills shown.
TF-IDF Importance Bar Chart	Keywords Tab	Horizontal bar chart of top 10 JD keywords by TF-IDF weight, colored green (matched) or red (missing).
Progress Bars (Custom HTML)	Overview, ATS Tabs	CSS-based animated progress bars for section scores, ATS dimensions, and keyword coverage.

7.3 Sentence-Level Match Highlighting

get_sentence_matches() provides a granular view of which specific resume sentences best align with specific JD sentences. This uses SBERT to encode all sentence pairs, compute a pairwise similarity matrix, and surface the top-N matches. Both resume and JD sentences are deduplicated to avoid redundant pairs.

Each match card shows the resume sentence, the JD sentence it aligns with, and the similarity percentage. This helps users understand which experience descriptions resonate with the JD and which do not.

7.4 AI Resume Rewriter

The AI Rewriter feature invokes rewrite_resume_with_openrouter(), which connects to OpenRouter's free-tier API. The system prompt instructs the model to act as an Executive Resume Writer and rewrite the professional summary and bullet points to incorporate missing keywords naturally — without hallucinating credentials.

Model Fallback Chain: The function iterates through six free LLM backends in order of preference, with a 2-second pause between retries:

- qwen/qwen3-coder:free
- google/gemma-4-31b-it:free
- meta-llama/llama-3.3-70b-instruct:free
- google/gemma-3-27b-it:free
- nousresearch/hermes-3-llama-3.1-405b:free
- nvidia/nemotron-3-super-120b-a12b:free

8. Limitations, Improvements, and Conclusion

8.1 Known Limitations

The current implementation has several documented limitations that affect accuracy or applicability:

Area	Limitation
PDF Extraction	pdfplumber cannot extract text from image-based (scanned) PDFs. No OCR fallback is implemented.
Skill Detection Precision	Skill matching is limited to the 270+ predefined taxonomy. Niche or emerging skills not in the taxonomy are silently missed.
NER Accuracy	spaCy's en_core_web_sm is a small general-purpose model. Company and person detection can mislabel proper nouns from skill names as organizations.
Education Scoring	Education section score is binary. It does not differentiate between degree levels (B.Tech vs. PhD) or field of study relevance.
ATS Simulation	The ATS scorer approximates real ATS behavior using heuristics. Different ATS platforms (Workday, Greenhouse, Lever) have proprietary parsing rules that cannot be fully replicated.
AI Rewriter Reliability	The AI rewriter depends on free-tier OpenRouter models which have rate limits. All six fallback models may fail simultaneously during peak usage.
Language Support	The entire pipeline assumes English-language resumes and job descriptions. Non-English input will produce unreliable results.

8.2 Recommended Improvements

The following enhancements would significantly improve the system's accuracy and usability:

- OCR Integration: Add pytesseract or AWS Textract fallback for image-based PDF extraction.
- Dynamic Skill Taxonomy: Replace the static skills.py with a database-backed taxonomy that can be updated without code changes, and add an admin UI for skill curation.
- Fine-tuned Embeddings: Replace the general-purpose all-MiniLM-L6-v2 model with a domain-specific model fine-tuned on resume-JD pairs.
- Degree Level Awareness: Expand education scoring to compare degree levels (BS < MS < PhD) and field-of-study relevance.
- Batch Analysis: Add support for uploading multiple resumes against a single JD for recruiter-side batch screening with ranking.
- Authenticated API Backend: Replace OpenRouter free-tier with a paid, authenticated API for reliable AI rewriting without rate limit failures.
- User Accounts and History: Add session persistence so users can compare multiple analyses and track resume improvement over time.

8.3 Conclusion

ResumeIQ is a well-engineered, production-grade NLP application that addresses a genuine and widespread problem: the disconnect between how resumes are written and what ATS systems and recruiters are looking for. Its multi-signal approach — combining SBERT semantics, TF-IDF keyword analysis, taxonomy-based skill matching, and ATS heuristics — produces more nuanced and reliable results than any single technique alone.

The codebase demonstrates strong software engineering practices: modular separation of concerns (app.py / utils.py / skills.py), defensive programming patterns (false positive filtering, score guards, model caching), calibrated thresholds backed by documented reasoning, and progressive enhancement through the AI rewriting feature.

The most impactful near-term improvements are OCR support for scanned PDFs and a dynamic, database-backed skill taxonomy. These two changes would eliminate the two most common failure modes and significantly expand the tool's real-world applicability.